

Phase-Varying Neural Potential Functions for Reactive Robot Motions

Ahmet Tekden¹

Dimitrios Kanoulas²

Aude Billard³

Yasemin Bekiroglu^{1,2}

Abstract—Dynamical systems (DS) methods for Learning-from-Demonstration (LfD) enable stable, continuous policies from few demonstrations. While first-order DS work well when each state maps to a unique velocity, they struggle with tasks involving intersections or repeated state visits. Existing solutions either rely on second-order dynamics, where velocity is used to disambiguate motion near intersections but may be insufficient when similar position-velocity pairs lead to different motions, or phase variables with open-loop progression, which reduce robustness to perturbations. We propose Phase-varying Neural Potential Functions (PNPF), an LfD framework that represents dynamical systems through phase-conditioned scalar potential functions, whose gradients generate the desired vector fields. The potential function additionally provides a measure of task progress, which we use to define a new phase variable that can be inferred directly from state evolution. This enables reactive control while disambiguating repeated states without relying on velocity or open-loop phase progression. PNPF generalizes across point-to-point, periodic, and 6D tasks, outperforms prior methods on trajectories with intersections, and demonstrates robust real-time performance under disturbances.

I. INTRODUCTION

Learning-from-Demonstration (LfD) enables robots to acquire task policies directly from demonstrations. A central challenge is generating motions that both follow demonstrations and remain reactive to perturbations. Dynamical systems (DS)-based approaches are attractive because they produce stable, continuous policies from few demonstrations [1]–[4]. However, they face a trade-off between modeling capability and reactivity. Time- or phase-dependent methods can represent tasks where the same state is revisited at different stages, but rely on open-loop progression and recover poorly after perturbations [1], [2]. In contrast, state-based DS methods react directly to the current state [3], [5]–[9], but require velocity or higher-order state information to handle intersections, making them sensitive to disturbances and ambiguous when similar position-velocity pairs should lead to different future motions (Fig. 1).

We address this problem with *Phase-varying Neural Potential Functions* (PNPF), a reactive motion-generation framework based on potential functions conditioned on a

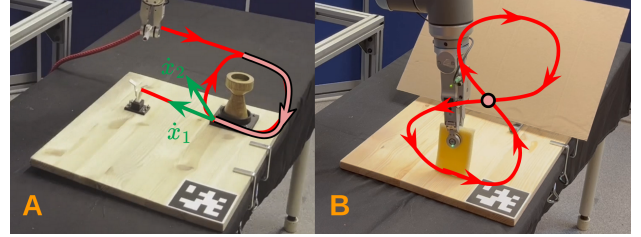


Fig. 1: Tasks where the robot revisits similar states: (A) point-to-point knotting, where identical motion segments branch into different actions $\hat{x}_1, \hat{x}_2$ from the same position-velocity state, and (B) 8-shaped periodic wiping. Our method ensures reactive motion generation in both cases.

novel phase variable. Rather than directly modeling the vector field, PNPF represents the dynamical system through a scalar potential function whose gradient generates the desired motion. From a geometric representation perspective, the resulting scalar field provides a continuous representation of motion over the robot state space, with its gradient defining the local motion direction. This also allows estimating phase directly from state progression, enabling the system to revisit similar states while remaining reactive to perturbations.

PNPF combines two energy functions: (i) a nominal energy, which encodes the desired task behavior and decreases monotonically with task progress; and (ii) a safety energy, which discourages leaving the boundary of demonstrated data, with this boundary estimated in a data-driven manner from the visited states. The nominal energy provides a scalar measure of task progress, enabling estimation of phase directly from the state and thereby resolving the ambiguity of repeated state visits without relying on velocity. The safety energy attracts the system back toward the region supported by demonstrations, encouraging recovery when perturbed while preserving compliant behavior within that region. Together, these two terms yield a motion-generation framework that remains reactive while handling point-to-point, periodic, and full 6D tasks.

Our approach is most closely related to state-based DS methods, movement primitives, and learned potential-function formulations [8], [10], [11]. In contrast to second-order DS methods, PNPF does not rely on velocity to disambiguate repeated states; in contrast to time-dependent motion primitives, it maintains reactivity through state-dependent estimation of phase; and in contrast to prior learned potential formulations, it explicitly handles repeated state visits through phase conditioning. More specifically, representing the dynamical system through a potential function provides a scalar quantity in addition to its gradient. In PNPF, the

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation, the Chalmers AI Research Center (CHAIR) and the Chalmers Gender Initiative for Excellence (Genie). D. Kanoulas was supported by UKRI FLF [MR/V025333/1] (RoboHike). A. Billard was supported by the euROBIN (European Robotics and AI Network) project under grant agreement No. 101070596. ¹Department of Electrical Engineering, Chalmers University of Technology, SE-412 96 Gothenburg, Sweden. ²Department of Computer Science, University College London, WC1E 6BT London, U.K. ³Learning Algorithms and Systems Laboratory (LASA), Ecole Polytechnique Fédérale de Lausanne (EPFL), 1015 Lausanne, Switzerland Email: tekden@chalmers.se

¹This work is based on a published IEEE RA-L paper, which provides the complete stability analysis and additional methodological details. Project page: <https://fzaero.github.io/PNPF/>

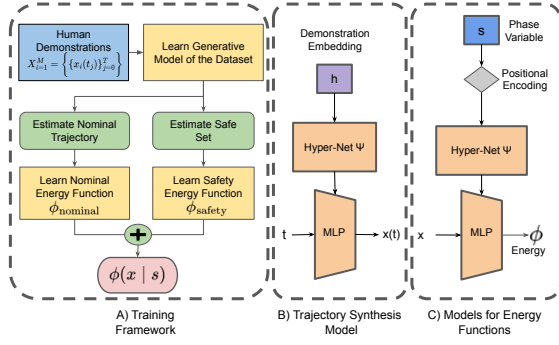


Fig. 2: Overview of PNPf. (A) Construction of the phase-varying potential $\phi(x | s)$. (B,C) Network architectures for trajectory synthesis and energy estimation.

nominal energy is constructed to decrease with task progress, allowing this scalar value to define the phase used to condition the potential function.

The main contributions are: (i) a reactive motion-generation framework based on phase-varying neural potential functions for point-to-point and periodic tasks; (ii) a novel phase variable that resolves repeated-state ambiguity while preserving reactivity; and (iii) compatibility with configuration-space, task-space, and full 6D motions. We evaluate the method on 2D handwriting and 6D real-robot tasks and show improved robustness and performance over prior reactive motion-generation baselines.

II. MOTION GENERATION WITH POTENTIAL FUNCTIONS

We represent motion as the gradient flow of a scalar-valued potential function over the state space. Intuitively, the potential defines an energy landscape in which desired task behavior corresponds to descending toward lower-energy regions. This perspective is appealing for reactive motion generation because it directly yields a dynamical system: rather than predicting trajectories in open loop, the robot continuously updates its motion by following the energy gradient. Formally, given state $x \in \mathbb{R}^d$ and phase variable s , the policy is

$$\dot{x} = -\nabla_x \phi(x | s), \quad (1)$$

where $\phi(x | s)$ is the potential function. This scalar representation provides both a local motion direction through its gradient and a scalar quantity that can be structured to reflect task progression. Conditioning on s allows the same state to correspond to different desired motions at different stages of the task, while preserving the closed-loop nature of the controller. Figure 2 summarizes the proposed architecture, while Figure 3 illustrates the resulting energy landscape and its phase-dependent behavior.

Our key idea is to construct $\phi(x | s)$ from two complementary energy functions. The *nominal energy* captures task progression by assigning lower values to states that lie further along the demonstrated motion. The *safety energy* captures whether a state lies within the region supported by demonstrations, and becomes active when the robot moves outside this region. Together, these energies provide an intuitive decomposition of behavior: the nominal term

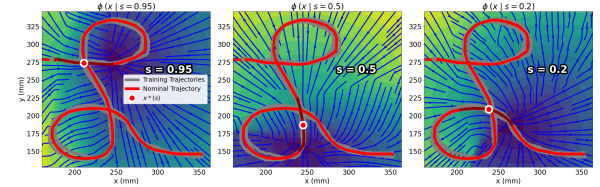


Fig. 3: Phase-varying potential functions. The same spatial state can correspond to different local descent directions at different phases of the task. This allows the system to revisit similar states while maintaining reactive, closed-loop motion generation.

encodes *task progression*, while the safety term determines *how the system should recover* when perturbed.

Constructing Nominal and Safety Energy Functions:

Given M demonstrations $X_{i=1}^M = x_i(t_0), \dots, x_i(t_T)$, we first learn a continuous trajectory synthesis model that captures the variation across the demonstrated trajectories. Each demonstration is represented by a learnable latent embedding, optimized jointly with a decoder-only neural network [12]. Sampling from the convex hull of these embeddings then allows us to generate trajectories that interpolate the demonstrated variations.

These synthesized trajectories serve two purposes. First, they are used to estimate a *nominal trajectory* $x^*(t)$ by selecting the generated trajectory with the lowest total DTW distance to the demonstrations. Second, we generate a large number of interpolated trajectories and use the states along them to densely approximate the demonstration-supported region $\mathcal{C}$. We represent this region using a signed distance function $\text{SDF}(x)$, where $\text{SDF}(x) \leq 0$ denotes states inside the support region and $\text{SDF}(x) > 0$ denotes states outside.

The safety energy is defined directly from the signed distance field:

$$\phi_{\text{safety}}(x) = \text{relu}(\lambda_{\text{safety}}(\text{SDF}(x))), \quad (2)$$

where λ_{safety} smooths the transition near the boundary. Thus, $\phi_{\text{safety}}(x) = 0$ inside the demonstrated support region and increases with distance outside it, causing its gradient to pull the system back toward demonstrated states.

The nominal energy is defined from the remaining arc length of the nominal trajectory. For the discretized nominal trajectory $\{x_j^*\}_{j=0}^N$,

$$\phi_{\text{nominal}}(x_k^*) = \sum_{j=k}^{N-1} \|x_{j+1}^* - x_j^*\|. \quad (3)$$

By construction, ϕ_{nominal} is non-negative, decreases monotonically as the system progresses along the nominal trajectory, and reaches zero at its end. It therefore provides a scalar measure of task progress associated with the robot state. For states away from the nominal trajectory, the energy is approximated by projecting the state onto the nearest point on x^* .

The nominal and safety energies are combined to obtain the potential function

$$\phi(x) = \phi_{\text{nominal}}(x) + k_{\text{safety}} \phi_{\text{safety}}(x), \quad (4)$$

where $k_{\text{safety}} > 0$ weights the safety term. The gradient of this potential therefore combines progression along the nominal

TABLE I: Method performance across different datasets.

Dataset	Method	DTWD	FD	FP Error	Accuracy
LASA (cm)	PNPF	1.81	3.38	0.02	1.00
	CONDOR	2.10	4.14	0.10	1.00
	NODE	2.12	4.33	1.82	0.76
LAIR (mm)	PNPF	4.82	11.75	5.04	1.00
	CONDOR	7.62	17.43	7.91	0.96
CHAR (mm)	PNPF	2.96	8.44	2.00	1.00
	CONDOR	17.67	45.56	3.25	1.00
Char-Periodic (mm)	PNPF	7.77	14.30	NA	NA
	Naive	10.70	24.28	NA	NA
Dataset	Method	DTWD-P (cm)	DTWD-O (rad)	FP Error (cm)	F0 Error (rad)
RoboTasks	PNPF	2.19	0.08	0.25	0.17
	NODE	2.62	0.10	1.51	0.13

behavior with recovery toward the demonstration-supported region.

Phase-Varying Potential Functions: The potential function described above can model many tasks effectively, but it cannot represent behaviors that revisit the same state at different stages of the motion. Since $\phi(x)$ assigns a single scalar value to each state, it cannot encode multiple energy values at the same location. To address this, we introduce a phase variable s that represents task progress.

Since ϕ_{nominal} decreases monotonically along the nominal trajectory, we define the phase using the nominal energy:

$$s_k = \phi_{\text{nominal}}(x_k^*). \quad (5)$$

We then construct local nominal and safety energies around the current phase. Given a phase window s_w , the nominal energy is constructed from the corresponding segment of the nominal trajectory, while the safety energy is constructed from demonstration-supported states within the corresponding phase interval. This yields the phase-varying potential

$$\phi(x | s) = \phi_{\text{nominal}}(x | s) + k_{\text{safety}} \Lambda_{\text{safety}}(s) \phi_{\text{safety}}(x | s), \quad (6)$$

where $\Lambda_{\text{safety}}(s)$ optionally increases the influence of the safety term later in the task.

The control policy and phase update are

$$\dot{x} = -\alpha \nabla_x \phi(x | s), \quad s \leftarrow \phi_{\text{nominal}}(x' | s), \quad (7)$$

where $\alpha > 0$ is a control gain and x' is the next state. The phase evolves according to the observed state progression rather than an open-loop temporal schedule. As a result, perturbations that alter task progression are reflected in the phase update, allowing the system to remain reactive while maintaining the appropriate stage of the motion.

The framework also extends to periodic and 6D motions. For periodic motions, the phase is represented using a sinusoidal embedding, allowing the potential function to vary periodically while preserving state-dependent phase estimation. For 6D motions, we follow [13], [14] and represent orientations in axis-angle form in the tangent space of a reference quaternion, allowing position and orientation to be modeled within the same potential-function formulation.

Neural Field Parameterization: We model $\phi_{\text{nominal}}(x | s)$ and $\phi_{\text{safety}}(x | s)$ using neural fields, which provide continuous and differentiable energy landscapes. To encourage

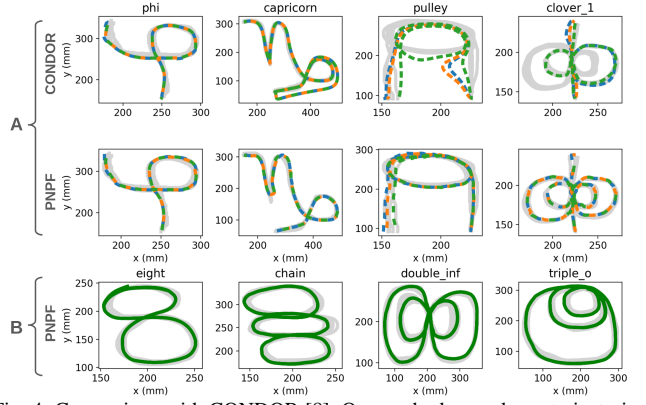


Fig. 4: Comparison with CONDOR [8]. Our method reproduces trajectories more faithfully and maintains correct motion through intersections.

smooth gradients with respect to x , we use a hypernetwork-conditioned [15] MLP: the phase variable s is position-encoded and used for conditioning, while the state x is provided directly. This provides expressive phase dependence while maintaining smooth variation with respect to the state for real-time control.

Stability: The potential function also provides a Lyapunov candidate for the closed-loop system. Assuming that the nominal and safety gradients do not cancel each other, $V(x, s) = \phi(x | s)$ decreases along the generated motion, with $\dot{V} = -\alpha \|\nabla_x \phi(x | s)\|^2 \leq 0$. Together with the closed-loop phase update, which monotonically decreases the phase toward zero, this establishes regional asymptotic convergence to the target set for trajectories initialized in the data-supported region.

III. SIMULATION-BASED EVALUATIONS

We evaluate PNPF on LASA [3] (2D), LAIR [8] (2D), and RoboTasks [14], [16] (6D), together with the CHAR and CHAR-Periodic datasets introduced in our work. We compare primarily against CONDOR [8], a state-based DS method that supports second-order state representations, and NODE [9].

LAIR and CHAR allow us to specifically evaluate phase conditioning against second-order state representations. In LAIR, velocity is sufficient to distinguish different motions when trajectories revisit similar positions. CHAR introduces a more challenging setting in which intersections are preceded by similar motion segments, such that similar position-velocity states require different onward actions. Consequently, second-order state information that is sufficient for LAIR cannot resolve the ambiguity in CHAR. CHAR-Periodic extends repeated-state evaluation to periodic motions.

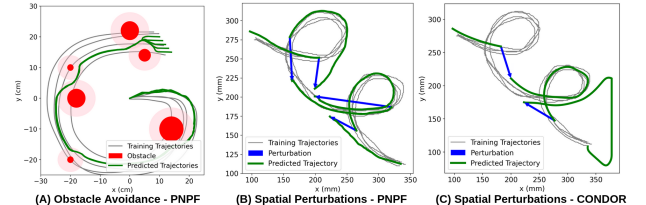


Fig. 5: PNPF remains reactive under obstacles and spatial disturbances, resuming motion without skipping task segments.

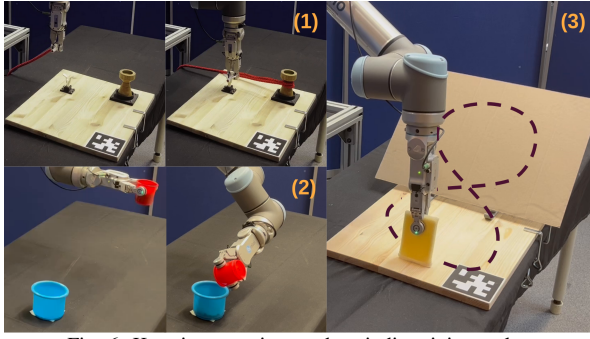


Fig. 6: Knotting, pouring, and periodic wiping tasks.

We evaluate geometric trajectory reproduction using DTW distance (DTWD) and Fréchet distance (FD), together with final-position error and task accuracy; for RoboTasks, we additionally evaluate orientation errors. All models are trained and evaluated with three random seeds.

Across all datasets, PNPf achieves the highest overall performance (Table I). The distinction between phase and second-order state representations is particularly apparent when comparing LAIR and CHAR. On LAIR, both PNPf and CONDOR can represent the demonstrated motions, although PNPf produces more consistent results across seeds. On CHAR, where similar position-velocity states require different continuations, PNPf reduces DTWD by approximately 80% compared with CONDOR. As illustrated in Fig. 4, CONDOR misses or mismodels trajectory segments around repeated states, whereas phase conditioning allows PNPf to maintain the appropriate task progression through these regions.

We additionally evaluate reactivity under obstacles and spatial disturbances. As shown in Figure 5, our method recovers smoothly and resumes the task without skipping or repeating motion segments, demonstrating the advantage of estimating phase directly from state progression.

IV. REAL-WORLD EXPERIMENTS

We evaluate the proposed method on three real-robot tasks: knotting, pouring, and periodic wiping (Fig. 6). Demonstrations are collected via kinesthetic teaching and time-aligned using DTW. We record six demonstrations for knotting and pouring, and two for periodic wiping. The knotting and periodic wiping tasks are modeled in task space using positions and quaternions, while pouring is modeled in joint space. Across all tasks, PNPf generates successful motions from multiple initial conditions while operating online at up to 50 Hz. For each task, we evaluate 10 different initial configurations; for periodic wiping, we additionally generate multiple motion cycles by initializing the phase accordingly. In all cases, the reproduced motions remain consistent with the demonstrations and successfully complete the intended behavior.

We further evaluate reactivity under environmental and spatial perturbations (Fig. 7). In the knotting task, we place obstacles in the workspace and test two additional disturbance settings: shifting the task frame and manually displacing the robot during execution. We also apply manual

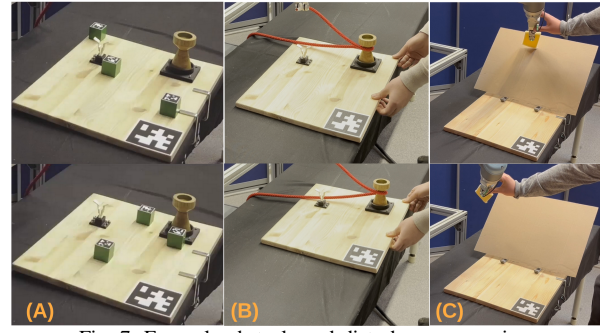


Fig. 7: Example obstacle and disturbance scenarios.

perturbations in the periodic wiping task. Across all tested cases, the robot adapts its motion while preserving task consistency and successfully completes the task. Notably, when the robot is pushed backward during knotting, causing the rope to untie from the cylinder, it correctly repeats the necessary part of the motion and re-ties the rope.

V. CONCLUSION

This work introduces a novel LfD framework for reactive motion generation based on phase-varying neural potential functions. The proposed method represents dynamical systems through scalar potential functions, whose gradients define the desired vector fields, while the nominal energy provides a measure of task progress used to define the phase variable. This allows the method to model dynamical systems that require progress-based parameterization while preserving reactivity to external perturbations. Simulation results show that our approach outperforms baselines on the evaluated datasets in terms of average DTWD and FD metrics. We demonstrate its effectiveness on real-robot control across three manipulation tasks, including both point-to-point and periodic motions. A current limitation is the tendency to oversmooth sharp corners due to ambiguities in the local energy formulation. Future work will refine the energy design to better preserve directional structure and improve robustness to demonstration artifacts, and incorporate visual and force feedback.

REFERENCES

- [1] S. Schaal, “Dynamic movement primitives—a framework for motor control in humans and humanoid robotics,” in *Adaptive motion of animals and machines*. Springer, 2006, pp. 261–280.
- [2] A. J. Ijspeert, J. Nakanishi, H. Hoffmann, P. Pastor, and S. Schaal, “Dynamical movement primitives: learning attractor models for motor behaviors,” *Neural computation*, vol. 25, no. 2, pp. 328–373, 2013.
- [3] S. M. Khansari-Zadeh and A. Billard, “Learning stable nonlinear dynamical systems with gaussian mixture models,” *IEEE Trans. Robot.*, vol. 27, no. 5, pp. 943–957, 2011.
- [4] A. Billard, S. Mirrazavi, and N. Figueroa, *Learning for adaptive and reactive robot control: a dynamical systems approach*. MIT Press, 2022.
- [5] N. Figueroa and A. Billard, “A physically-consistent bayesian non-parametric mixture model for dynamical system learning,” in *Conf. Robot. Learn. (CoRL)*, 2018, pp. 927–946.
- [6] N. Figueroa, S. Faraji, M. Koptev, and A. Billard, “A dynamical system approach for adaptive grasping, navigation and co-manipulation with humanoid robots,” in *IEEE Int. Conf. Robot. Autom. (ICRA)*, 2020, pp. 7676–7682.
- [7] F. Khadivar, I. Lauzana, and A. Billard, “Learning dynamical systems with bifurcations,” *Robot. Auton. Syst.*, vol. 136, p. 103700, 2021.

- [8] R. Pérez-Dattari and J. Kober, “Stable motion primitives via imitation and contrastive learning,” *IEEE Trans. Robot.*, 2023.
- [9] F. Nawaz, T. Li, N. Matni, and N. Figueroa, “Learning complex motion plans using neural odes with safety and stability guarantees,” in *IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 17 216–17 222.
- [10] S. M. Khansari-Zadeh and O. Khatib, “Learning potential functions from human demonstrations with encapsulated dynamic and compliant behaviors,” *Auton. Robots*, vol. 41, pp. 45–69, 2017.
- [11] M. A. Rana *et al.*, “Learning reactive motion policies in multiple task spaces from human demonstrations,” in *Conf. Robot. Learn. (CoRL)*, 2020, pp. 1457–1468.
- [12] J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove, “Deepsdf: Learning continuous signed distance functions for shape representation,” in *IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 165–174.
- [13] A. Ude, B. Nemec, T. Petrić, and J. Morimoto, “Orientation in cartesian space dynamic movement primitives,” in *IEEE Int. Conf. Robot. Autom. (ICRA)*, 2014, pp. 2997–3004.
- [14] S. Auddy, J. Hollenstein, M. Saveriano, A. Rodríguez-Sánchez, and J. Piater, “Continual learning from demonstration of robotics skills,” *Robot. Auton. Syst.*, vol. 165, p. 104427, 2023.
- [15] D. Ha, A. M. Dai, and Q. V. Le, “Hypernetworks,” in *Int. Conf. Learn. Represent. (ICLR)*, 2017.
- [16] S. Auddy, J. Hollenstein, M. Saveriano, A. Rodríguez-Sánchez, and J. Piater, “Scalable and efficient continual learning from demonstration via a hypernetwork-generated stable dynamics model,” 2024.